

คำถามวิเคราะห์และสรุปผลเชิงลึก (Analysis Questions)

การออกแบบและเปรียบเทียบระบบประวัติเบราว์เซอร์: Two-Stack vs ArrayList

ข้อ 1: Invariant ของ Back Stack และ Forward Stack คืออะไร?

- Back Stack Invariant:** ข้อมูลบนสุดของ Stack (Top) จะต้องเป็น "หน้าเว็บที่เพิ่งเปิดล่าสุดก่อนหน้าปัจจุบันเสมอ" จัดเก็บแบบย้อนเวลาตามลำดับ LIFO (Last-In, First-Out)
- Forward Stack Invariant:** ข้อมูลบนสุดของ Stack (Top) จะต้องเป็น "หน้าเว็บถัดไปที่เพิ่งถูก Back ออกมาเสมอ" จัดเก็บแบบเดินหน้าตามลำดับที่เคยถอยออกมา
- Current Page Invariant:** ระบบต้องมีหน้าปัจจุบัน (`currentPage`) ได้เพียง 1 หน้าเสมอ และหน้านั้นจะต้องไม่อยู่ซ้ำซ้อนใน Stack ทั้งสองพร้อมกัน

ข้อ 2: เหตุใด Forward History ต้องถูกล้างเมื่อ Visit หน้าใหม่?

- ป้องกันประวัติแตกกิ่งก้าน (Prevent Branching Timeline):** การนำทางของเว็บเบราว์เซอร์ถูกกำหนดให้เป็น เส้นตรง (Linear Timeline) หากผู้ใช้กดย้อนกลับแล้วเข้าชมหน้าเว็บใหม่ เส้นทางเดิมข้างหน้าจะกลายเป็น "อนาคตคู่ขนานที่ไม่เกี่ยวข้องกัน เส้นทางใหม่" จึงต้องตัดทิ้ง
- ประสบการณ์ผู้ใช้ (UX Consistency):** เมื่อเปิดหน้าใหม่ การกดปุ่ม Forward กันจะต้องไม่มีผลการทำงาน เพราะประวัติข้างหน้าถูกเขียนทับด้วยเส้นทางใหม่อย่างสมบูรณ์แล้ว

ข้อ 3: Two-Stack และ ArrayList แตกต่างกันอย่างไรเมื่อเปิดหน้าใหม่หลัง Back?

ประเด็นเปรียบเทียบ	Two-Stack Method	ArrayList & Current Index Method
กลไกการล้าง Forward	เรียก <code>forwardStack.clear()</code> ซึ่งเป็นการ Reset Head/Tail Pointer ของ <code>ArrayDeque</code> ทันที	เรียก <code>subList(idx+1, size).clear()</code> ซึ่งต้องวนวน Object ที่อยู่ด้านหลังทีละตำแหน่ง
การใช้ Memory	ปล่อย Object ให้ Garbage Collector กันที ไม่กระทบขนาดโครงสร้าง Back Stack	ต้องจัดการขยับลดขนาด Dynamic Array และอาจเกิด Overhead ของ Array Capacity
ความซับซ้อนของโค้ด	ตรงไปตรงมา จัดการแยกตัวแปรและโครงสร้างชัดเจน	ต้องคอยคำนวณและตรวจสอบ Index อย่างระมัดระวังเพื่อไม่ให้เกิด OutOfBounds

ข้อ 4: Amortized $O(1)$ แตกต่างจาก Worst-case $O(n)$ อย่างไร?

- Worst-case $O(n)$:** คือกรณีที่การทำงานในรอบนั้นต้องใช้เวลาแปรผันตามจำนวนข้อมูล n ตัวเต็มๆ เสมอ เช่น การลบข้อมูลครึ่งหลังใน ArrayList ด้วย `subList().clear()` หรือการคัดลอก Array ใหม่ทั้งหมดเมื่อพื้นที่เต็ม
- Amortized $O(1)$:** คือ "เวลาเฉลี่ยต่อการหนึ่งการกระทำ" เมื่อทำซ้ำหลายๆ ครั้ง แม้จะมีบางจังหวะที่เกิดเหตุการณ์ซ้ำเป็น $O(n)$ เป็นครั้งคราว (เช่น การขยายขนาด Array) แต่เมื่อเฉลี่ยต้นทุนรวมกับการทำงานปกติที่เร็ว $O(1)$ ส่วนใหญ่แล้ว จะได้ค่าเฉลี่ยออกมาเป็น $O(1)$ ต่อรอบเสมอ

ข้อ 5: วิธีใดเหมาะกับ Browser History มากกว่ากัน?

สรุป: Two-Stack Method (ใช้ ArrayDeque) เหมาะสมกว่าอย่างชัดเจน

- **Time Complexity สม่่าเสมอ:** การทำงานหลักอย่าง visit, back, forward ทำงานแบบ $O(1)$ อย่างแท้จริงและสม่ำเสมอตลอดเวลา
- **ตรงกับลักษณะการใช้งานจริง (Natural Mapping):** พฤติกรรม Back/Forward เป็นการดึงข้อมูลหัวแถวเข้า-ออกแบบ LIFO โดยธรรมชาติ การใช้ Stack จึงไม่ต้องสิ้นเปลืองเวลา Shift ข้อมูล หรือคำนวณ Index เหมือน ArrayList